ChatDBG-MTP: Extending AI-Assisted Debugging with Post-Mortem Analysis, Repository-Aware Context, and Automated Repair

Siddhi Tiwari
Department of Computer Science & Engineering
[Your Institution]
City, Country

siddhitiwari0109@gmail.com

***Abstract*—Debugging is one of the most time-consuming activities in software development. Existing AI-assisted debugging tools, including the original ChatDBG, require an active debugger session, limiting their use to crashes reproducible in a live environment. In this work we present ChatDBG-MTP, an extended version of ChatDBG that introduces five major contributions: (1) post-mortem analysis that runs on crash log files without a live process, (2) a zero-friction exception hook that auto-triggers analysis on any unhandled exception, (3) repository-aware context that follows import graphs to provide the LLM with a broader view of the codebase, (4) git-aware context that surfaces recent commit history, blame information, and diffs to support regression detection, and (5) automated fix application and test generation that complete the full repair cycle. We evaluate ChatDBG-MTP on 8 benchmark programs across four configuration variants. Our results show that adding repository context improves root-cause keyword coverage from 57% to 77% (+20 pp), fix correctness from 50% to 75% (+25 pp), and test generation from 12% to 75% (+63 pp). Combining repository and git context achieves the best overall accuracy (89% keyword coverage, 88% fix correctness) with a modest cost increase of 38%.**

***Index Terms*—debugging, large language models, post-mortem analysis, automated program repair, fault localization, git-aware debugging, test generation**

I. Introduction

Modern software crashes in many contexts where a live debugger is unavailable: production environments, continuous integration pipelines, remote machines, and crash reports submitted by end users. Even when a debugger is available, AI assistants such as ChatDBG [1] only analyze the immediate stack trace, missing broader codebase context that could accelerate diagnosis.

We identify three key gaps in existing AI-assisted debugging:

1) **Availability** — tools require a live debugger session, excluding the majority of real-world crashes captured in logs.
2) **Context** — analysis is limited to the call stack, ignoring imported modules, class hierarchies, and recent code changes that frequently contain the root cause.
3) **Completeness** — tools explain bugs but do not close the repair loop: applying the fix and writing a regression test.

ChatDBG-MTP addresses all three gaps through the following contributions:

- A **post-mortem analysis** mode (`chatdbg --analyze crash.log`) that parses Python tracebacks from text files and runs LLM analysis without any running process.
- A **zero-friction exception hook** (`chatdbg.install()`) that registers a `sys.excepthook` so every unhandled exception is automatically analyzed.
- A **repository context** mechanism (`--repo`) that traverses the import graph from crashed frames to supply relevant source files.
- A **git context** mechanism that automatically includes `git log`, `git blame`, and `git diff` output for each crashed file.
- An **automated repair** pipeline using two LLM tools: `apply_fix` and `generate_test`.
- An **evaluation harness** for systematic comparison of configuration variants.

II. Background and Related Work

A. ChatDBG

ChatDBG [1] integrates an LLM into the Python `pdb`/`ipdb` debugger and the native GDB/LLDB debuggers. When the user types `why`, ChatDBG sends the stack trace, error message, and local variable values to the LLM, which can call back into the debugger via registered functions (`debug`, `info`, `slice`). ChatDBG uses `litellm` to support multiple model providers.

B. Automated Program Repair

APR tools such as GenProg [2], Prophet [3], and Angelix [4] use search or symbolic analysis to generate patches. Recent LLM-based approaches (e.g., AlphaRepair, ChatRepair) prompt models with the buggy code and test suite. Unlike these tools, ChatDBG-MTP operates at debugging time on a

crashed program, without requiring a pre-existing test suite or formal specification.

C. Fault Localization

Spectrum-based fault localization (e.g., Tarantula [7], Ochiai) ranks suspicious statements based on test pass/fail patterns. Our approach is complementary: rather than ranking statements, we provide the LLM with rich context and rely on its reasoning to identify the root cause.

D. LLM Code Understanding

Recent work shows that LLMs can reason about code at multiple levels of abstraction [5]. Providing additional context—function signatures, docstrings, related code—consistently improves accuracy [6]. Our `--repo` feature operationalizes this by automatically surfacing the most relevant context via import graph traversal.

III. System Design

A. Architecture Overview

ChatDBG-MTP extends the original ChatDBG pipeline with a new post-mortem pathway that runs independently of any live debugger. The pipeline parses the traceback, builds source/repo/git context, constructs a prompt, and invokes the LLM assistant, which may call back into `apply_fix` and `generate_test`. The interactive pdb mode and post-mortem mode share the same `Assistant` class, listeners, and YAML logging infrastructure.

B. Post-Mortem Analysis

The `postmortem` package contains three components:

Parser (`parser.py`): Parses Python tracebacks into a `CrashReport` dataclass containing `FrameInfo` records (filename, line number, function, source line) and the exception type/message. Handles chained exceptions, log files with embedded tracebacks, and exceptions without messages.

Context builder (`context.py`): For each user-code frame, reads `context_lines` lines around the crash point from disk using `linecache`. Library frames (`site-packages`, stdlib) are filtered out.

Analyzer (`analyze.py`): Orchestrates the pipeline, creates the `Assistant` with `apply_fix` and `generate_test` tools, runs the LLM query, and logs results.

C. Exception Hook

`chatdbg.install()` registers a `sys.excepthook` that: (1) lets Python print the original traceback; (2) formats the traceback as text using `traceback.format_exception`; (3) calls `analyze_crash_text()` in memory; (4) silently skips `SystemExit` and `KeyboardInterrupt`; and (5) catches all ChatDBG errors so they never mask the original exception.

D. Repository Context

`repo.py` traverses the local import graph starting from the files in the traceback. It collects all `.py` files under `--repo`, seeds with files directly referenced in the crash frames, then parses `import` and `from ... import` statements using the `ast` module to find locally-defined dependencies to depth 2. Discovered files are formatted into the prompt within a token budget of $\approx 6{,}000$ tokens.

E. Git-Aware Context

`git_context.py` runs three `git` commands per unique user-code file via `subprocess`, as summarized in Table I. All commands are timeout-protected (10 s) and fail silently if git is unavailable.

TABLE I
GIT COMMANDS USED FOR EACH CRASHED FILE

Command	Purpose
`git log --oneline -8 <file>`	Recent commit history
`git blame -L <s>,<e> <file>`	Last author/date of crash lines
`git diff HEAD~1 -- <file>`	Changes in most recent commit

F. Automated Repair

Two LLM tool functions complete the repair cycle:

`apply_fix(filename, old_code, new_code)`: Shows a unified diff and prompts the user `[y/N]` before writing. Old-code matching uses line-ending-normalized comparison; CRLF is restored on write.

`generate_test(filename, test_code)`: Generates a pytest test that reproduces the original crash. The test is designed to fail against the buggy code and pass after the fix, creating a regression guard.

Both tools bypass `input()` prompts in eval mode (`CHATDBG_EVAL=1`), capturing proposed changes in memory for the evaluator to score.

IV. Evaluation

A. Experimental Setup

We evaluate ChatDBG-MTP on 8 benchmark programs covering common Python bug categories (Table II). We test four configuration variants: **Baseline** (traceback + source context), **+Repo** (+ import-graph files), **+Git** (+ git log/blame/diff), and **+Repo+Git** (all context). All experiments use GPT-4o via the OpenAI API.

B. Metrics

We measure: (1) **Exception Match** — does the response name the expected exception? (2) **Keyword Coverage (KW%)** — fraction of expected root-cause keywords found in the response; (3) **Fix Proposed** — was `apply_fix` called? (4) **Test Generated** — was `generate_test` called? (5) **Fix Correct** — does the proposed fix make the script exit with code 0? (6) **Cost** (USD); and (7) **Tokens** per analysis.

TABLE II
BENCHMARK PROGRAMS

ID	Bug Type	Exception
testme_zero_division	Loop variable starts at 0	ZeroDivisionError
sample_swapped_args	NumPy arguments swapped	TypeError
off_by_one_index	List access beyond bounds	IndexError
none_attribute_access	Method call on None	AttributeError
type_mismatch	String + integer addition	TypeError
key_error_dict	Missing dictionary key	KeyError
recursion_overflow	No base case in recursion	RecursionError
value_out_of_range	Invalid enum/range value	ValueError

C. Results

Table III shows aggregate metrics averaged across all 8 benchmarks.

TABLE III
AGGREGATE EVALUATION RESULTS (8 BENCHMARKS)

Variant	Exc	KW%	Fix	Test	FixOK	Cost	Tok
Baseline	88%	57%	75%	12%	50%	$0.019	1641
+Repo	100%	77%	100%	75%	75%	$0.024	2081
+Git	100%	72%	100%	50%	62%	$0.024	2069
+Repo+Git	100%	**89%**	100%	**88%**	**88%**	$0.026	2511

Key findings:

Repository context is the single most impactful feature, improving keyword coverage by 20 pp and fix correctness by 25 pp over baseline. This confirms that root causes frequently lie in imported modules not directly visible in the stack trace.

Git context alone improves recall modestly (+15 pp KW, +12 pp fix correctness), with its largest gains on regression-type bugs where the `git diff` identifies the introducing commit.

Combining both contexts achieves the best results across all metrics. The improvement is super-additive for test generation (12% $\rightarrow$ 88%), suggesting both context types are needed for the LLM to generate runnable tests.

Cost is modest and predictable: the full +Repo+Git variant costs on average $0.026 per analysis, a 38% increase over baseline for a 56% improvement in keyword coverage.

V. DISCUSSION

A. When Does +Repo Help Most?

Repository context provides the largest improvement on bugs where the root cause is in a helper function imported by the crashed module (e.g., `sample_swapped_args`, `none_attribute_access`). For self-contained bugs, the improvement is marginal. This suggests a heuristic: use `--repo` when the crash occurs in application code that imports project-defined modules.

B. When Does +Git Help Most?

Git context is most valuable for regression bugs introduced by a recent commit. In our benchmark, `off_by_one_index` and `recursion_overflow` showed the largest gains with +Git, consistent with bugs introduced by a code change visible in `git diff HEAD~1`. For bugs that have existed in the codebase for many commits, git context adds noise rather than signal.

C. Limitations

Static keyword matching is a proxy for correctness. Future work should use LLM-as-judge evaluation for semantic correctness.

Fix correctness verifies the script exits cleanly on the original input but does not guarantee absence of regressions on other inputs.

Source file availability: post-mortem analysis requires source files to be present at the paths recorded in the traceback.

Single-model evaluation: all experiments use GPT-4o. Future work should benchmark across Claude, Gemini, and open-source models.

VI. CONCLUSION

We presented ChatDBG-MTP, an extension of ChatDBG that enables post-mortem debugging from crash logs, enriches LLM context with import-graph traversal and git history, and closes the repair cycle with automated fix application and test generation. Our evaluation on 8 benchmarks demonstrates consistent, measurable improvements: adding repository and git context together raises root-cause keyword coverage from 57% to 89% and fix correctness from 50% to 88%, at a modest cost increase of 38%.

The evaluation harness introduced in this work enables systematic comparison of context strategies and model choices, providing a foundation for future research into AI-assisted program repair.

ACKNOWLEDGMENT

The author thanks the original ChatDBG team (Berger, Freund, Levin, van Kempen) for the open-source codebase on which this work builds.

REFERENCES

[1] E. D. Berger, S. Freund, K. Levin, and N. van Kempen, "ChatDBG: An AI-Powered Debugging Assistant," in *Proc. ACM Int. Conf. Foundations of Software Engineering (FSE)*, 2025.

[2] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, "GenProg: A Generic Method for Automatic Software Repair," *IEEE Trans. Softw. Eng.*, vol. 38, no. 1, pp. 54–72, 2012.

[3] F. Long and M. Rinard, "Automatic Patch Generation by Learning Correct Code," in *Proc. ACM SIGPLAN-SIGACT Symp. Principles of Programming Languages (POPL)*, pp. 298–312, 2016.

[4] S. Mechtaev, J. Yi, and A. Roychoudhury, "Angelix: Scalable Multiline Program Patch Synthesis via Symbolic Analysis," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, pp. 691–701, 2016.

[5] M. Chen, J. Tworek, *et al.*, "Evaluating Large Language Models Trained on Code," *arXiv preprint arXiv:2107.03374*, 2021.

[6] N. Nashid, M. Sintaha, and A. Mesbah, "Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, pp. 2450–2462, 2023.

[7] J. A. Jones and M. J. Harrold, "Empirical Evaluation of the Tarantula Automatic Fault-Localization Technique," in *Proc. IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, pp. 273–282, 2005.